

BINTANG HARI KAHONO

Selected Projects Portfolio

kahonokun@gmail.com | linkedin.com/in/kahonokun777 | github.com/harikahono



harikahono.github.io

Project Title

SEIRIS - Dynamic Equity Management Platform

Year

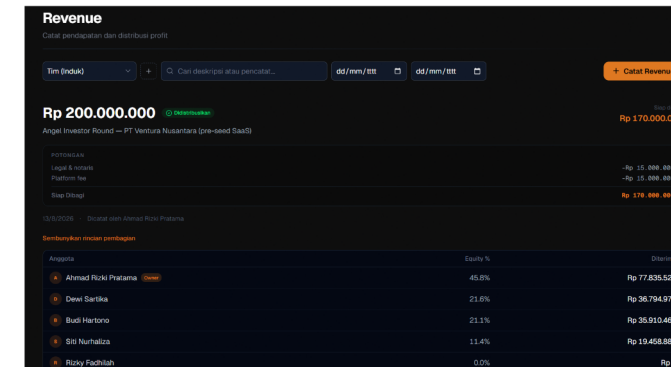
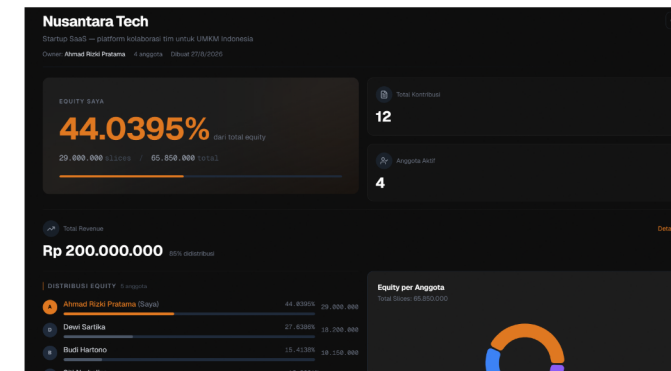
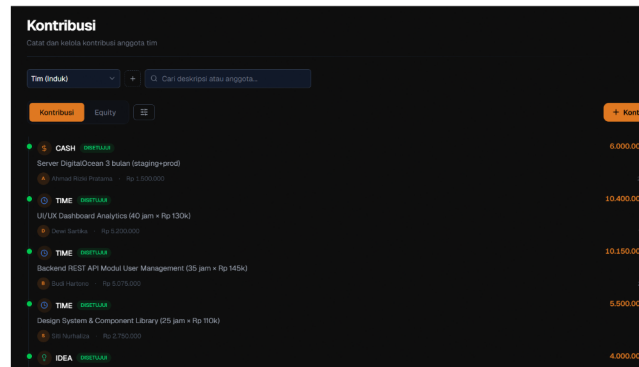
Mar 16, 2026 - Aug 27, 2026

Role

Full-stack Developer (solo, final-year academic project)

Link

seiris.codiroom.tech



A full-stack app automating equity allocation for student startup teams with the Slicing Pie model.

- Real-time collaboration with Pusher; UML and finite-state-machine design.
- Built backend, frontend, schema, and deployment independently.

What I learned: Turning a complex business model into code: the Slicing Pie contribution formula (cash x4, non-cash x2, sales commission) with nested pies and zero-loss roll-up, every rule guarded by four finite-state machines. A real production bug (duplicate projects on slow networks) taught me layered concurrency defense - DB unique constraint + rule validation + pessimistic row locking, documented as ADR-031.

Project Title

Z4 Foundation - Web3 Land Investment Platform

Year

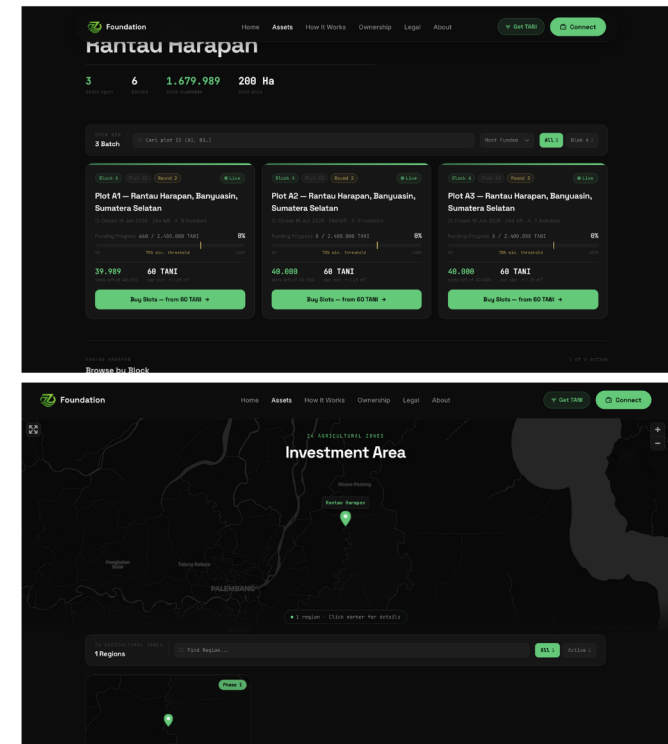
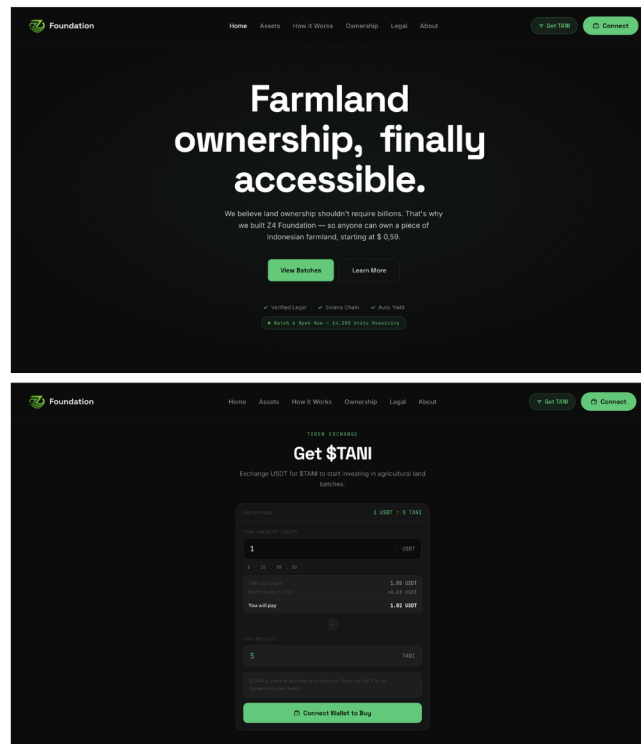
Mar 29, 2026 - May 23, 2026

Role

Frontend Developer (GROUP PROJECT, community Web3 pilot)

Link

z4foundation.io



A Solana platform fractionalizing agricultural land ownership for smaller investors.

- Investor-facing flows: wallet integration, listings, dashboards.
- Translated complex Web3 concepts into clearer product screens.

What I learned: *The biggest lesson was translating blockchain complexity into a user journey that feels safe and clear: SIWS wallet auth with nonce challenge, Anchor IDL integration (including Anchor 0.30+ naming breaking changes), PDA/ATA derivation on the frontend, and reconciling a rich business lifecycle with a simplified on-chain state machine. I also built a mock/demo mode so the product is presentable without a live backend, and learned that sensitive financial UX needs risk disclosure before any buy CTA.*

Project Title

PMB PKU-MI - New Student Admission System

Year

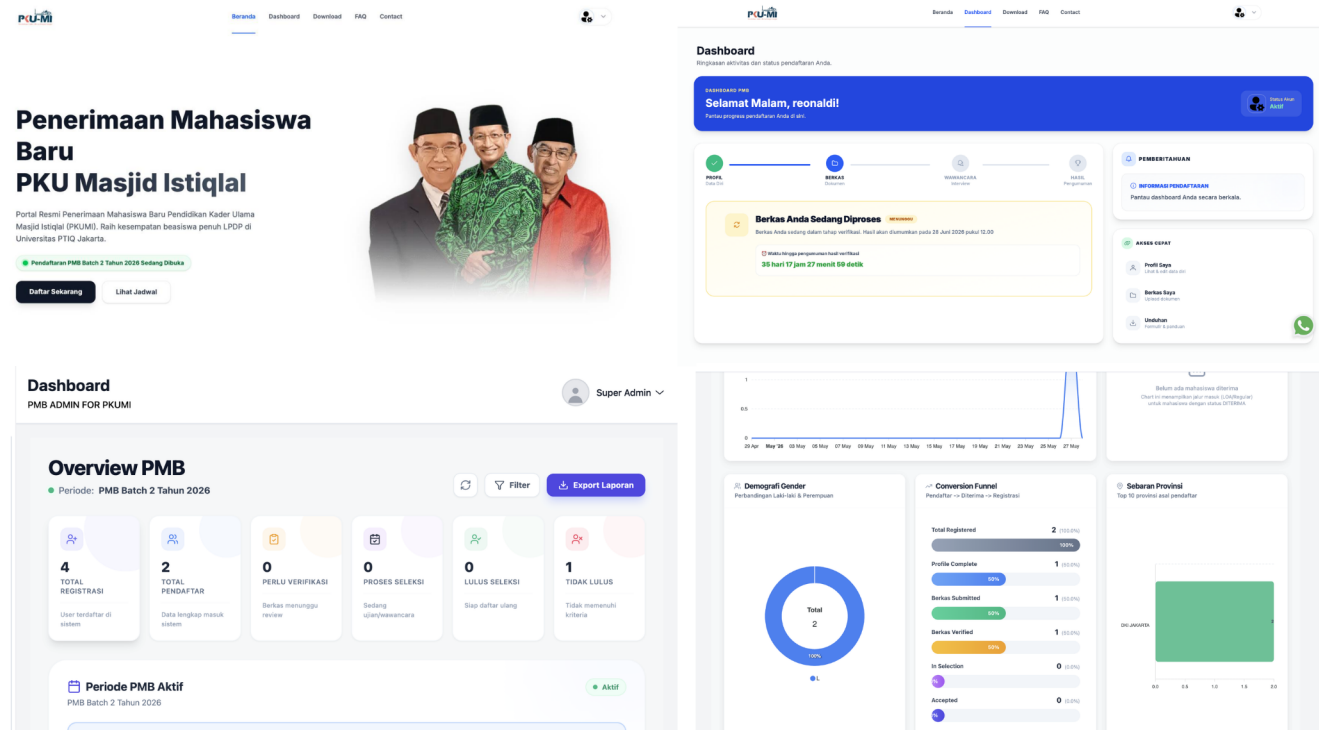
Oct 24, 2025 - Jul 18, 2026

Role

Frontend Developer (GROUP PROJECT, institutional, live)

Link

pmbpku.istiqlal.or.id



End-to-end S2/S3 admission platform for the Ulama Education Program at Istiqlal Mosque Jakarta.

- Applicant flows used by hundreds of applicants.
- Paperless ops: PDF invitations, LOA letters, Excel exports, RBAC.

What I learned: Solved a real Next.js App Router limitation - middleware cannot read localStorage - with a server-side auth proxy: HttpOnly cookie for SSR safety plus client token for API calls. Modeled a 12-state registration pipeline with revision loops and time-based auto-lock (60s polling), per-type file validation with inline preview, and client-side PDF generation for official documents.

Project Title

Ola-ATK - Stationery Retail SaaS

Year

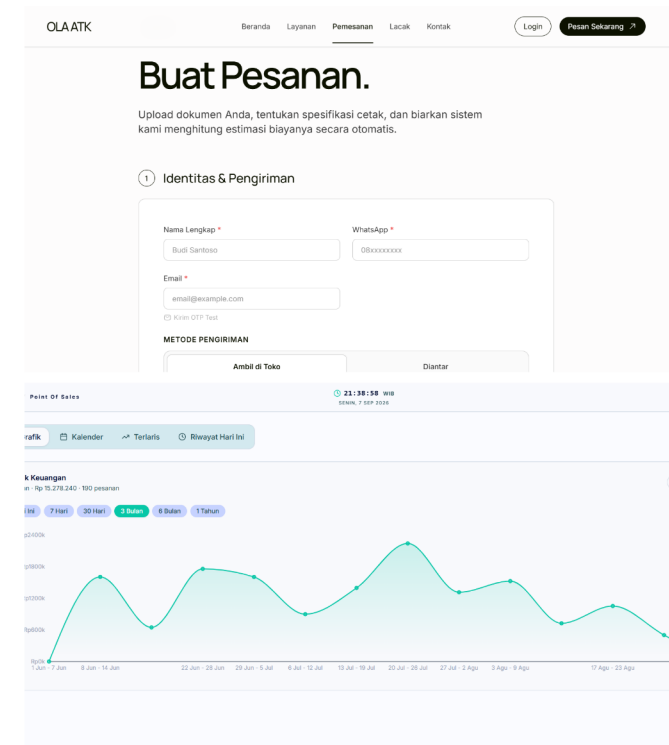
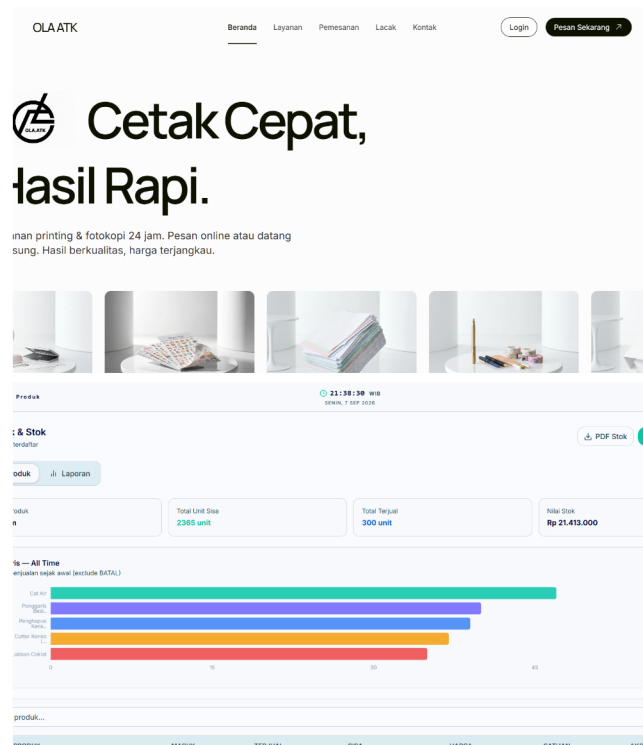
Mar 10, 2026 - Aug 24, 2026

Role

Full-stack Developer (solo project)

Link

ola.codiroom.tech



SaaS for stationery store operations: inventory, sales, admin, with a React dashboard.

- Backend models and APIs with Prisma/MySQL tied to a practical dashboard.
- Designed around real store operations.

What I learned: An idempotent two-step Midtrans payment (create-token with zero DB writes, confirm after payment, webhook as fallback) eliminated orphan order data - and taught me not to trust webhooks as the primary path. OTP email auth with real anti-abuse limits (10/hour, 60s cooldown, 5 verify attempts), plus turning a non-technical requirement into a scalable pricing scheme (gramasi, print-side, delivery) and browser-side BW/color PDF detection.

Project Title

wtf-bro - CLI Safety Net for AI Coding (ongoing)

Year

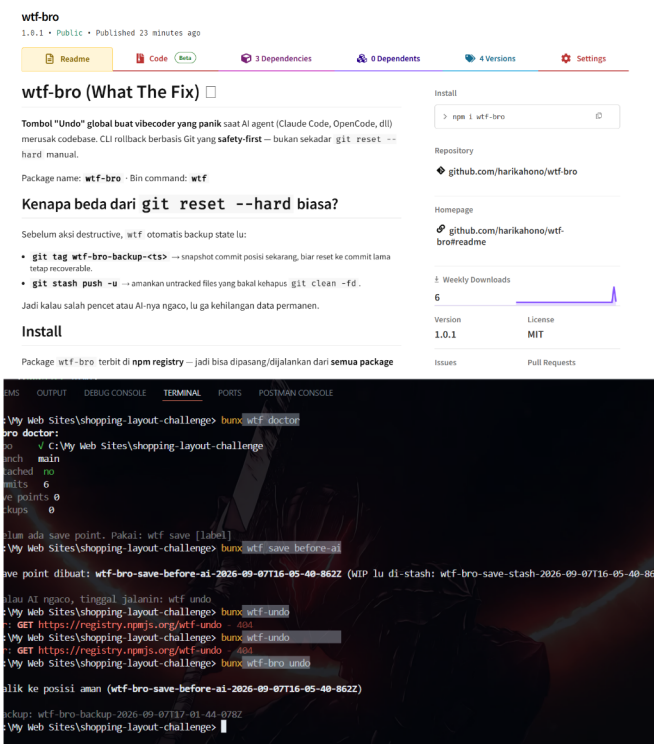
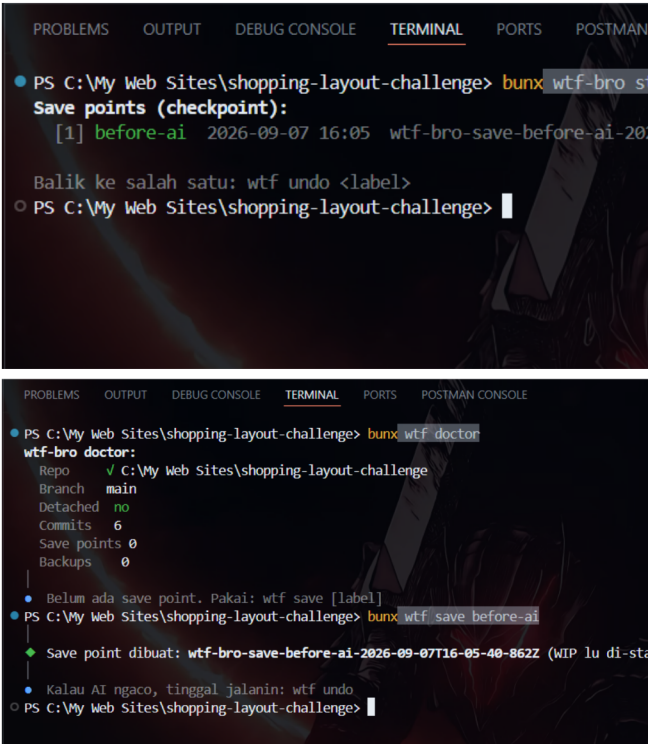
Sep 7, 2026 - Present

Role

Solo project, work in progress

Link

npmjs.com/package/wtf-bro



A CLI giving instant undo when an AI coding agent damages a codebase.

- Safety-first git rollback flows with backups and audit logging.
- Minimal CLI designed for panic recovery.

What I learned: The core lesson is safety-first design: backup before every destructive action (git tag + stash), undo that is idempotent (one save point covers many mistakes), and deliberate friction - retype the branch name - before irreversible resets. Publishing to npm taught me registry research, the files-field trap, and semver discipline; wtf init makes AI agents prepare save points but never roll back on their own.